SP-4 Phase D — Multi-Provider Parallel Search SDK (implementation plan)

Plan date: 2026-05-13 **Design:** `docs/superpowers/specs/2026-05-13-sp4-phase-d-design.md` **Parent SP:** `docs/superpowers/specs/2026-05-12-sp4-multi-provider-redesign.md` **Prerequisite:** Phase C complete (HEAD `6db79c50`).

6 tasks, ~20 steps. The SDK-side work (Tasks 1-3) + ActiveTrackerSection removal (Task 4) are autonomously executable. The Search-Result UI branch (Task 5) is also autonomous. Cancellation + backpressure unit tests (Task 6) ship in the same commit. Compose UI Challenges C32 + C33 are documented as KDoc rehearsal protocols; operator runs them on the gating emulator before next tag.

Task 1 — `MultiSearchEvent` sealed hierarchy + builder

The event vocabulary the streaming SDK method emits.

Files:

- Create: `core/tracker/client/src/main/kotlin/lava/tracker/client/MultiSearchEvent.kt`

Steps:

- [ ] **Step 1: Sealed hierarchy.**

```
sealed interface MultiSearchEvent {
    val providerId: String

    data class ProviderStart(
        override val providerId: String,
        val displayName: String,
    ) : MultiSearchEvent

    data class ProviderResults(
```

```
        override val providerId: String,
        val items: List<TorrentItem>,
    ) : MultiSearchEvent

    data class ProviderFailure(
        override val providerId: String,
        val reason: String,
        val cause: Throwable? = null,
    ) : MultiSearchEvent

    data class ProviderUnsupported(
        override val providerId: String,
    ) : MultiSearchEvent

    data class AllProvidersDone(
        override val providerId: String = "",
        val unified: UnifiedSearchResult,
    ) : MultiSearchEvent
}
```

- [ ] **Step 2: Builder helper.** Add a companion `fun UnifiedSearchResult.Companion.fromEvents(query, page, events): UnifiedSearchResult` — consume the event sequence, build the `providerStatuses` + `resultsByProvider` maps, run `DeduplicationEngine.deduplicate(...)`, return the assembled `UnifiedSearchResult`.

- [ ] **Step 3: Compile.**

Task 2 — Rewrite `LavaTrackerSdk.multiSearch` to parallel fan-out

Files:

- Modify: `core/tracker/client/src/main/kotlin/lava/tracker/client/LavaTrackerSdk.kt`

Steps:

- [ ] **Step 1: Extract per-provider logic** into a private `suspend fun runOneProvider(id, descriptor, request, page): ProviderOutcome` returning a sealed result (Success(items), Failure(reason), Unsupported, NotRegistered).

- [ ] **Step 2: Rewrite multiSearch body:**

```
suspend fun multiSearch(
    request: SearchRequest,
    providerIds: List<String>,
    page: Int = 0,
): UnifiedSearchResult = coroutineScope {
    val outcomes = providerIds.map { id ->
        async { id to runOneProvider(id, request, page) }
    }.awaitAll()
    // Build UnifiedSearchResult from the parallel outcomes (logic
        moved here from current sequential impl)
    val statuses = outcomes.map { (id, outcome) ->
        outcome.toStatus(id) }
    val resultsByProvider = outcomes.mapNotNull { (id, outcome) ->
        (outcome as? ProviderOutcome.Success)?.let { id to
        it.items }
    }.toMap()
    val displayNames = outcomes.associate { (id, outcome)
        -> id to outcome.displayName(id) }
    val deduped =
        DeduplicationEngine.deduplicate(resultsByProvider,
        displayNames)
    UnifiedSearchResult(
        query = request.query,
        page = page,
        items = deduped,
        totalPages = 1,
        providerStatuses = statuses,
    )
}
```

- [ ] **Step 3: Compile.**

Task 3 — Add `streamMultiSearch(...)` via channelFlow

Files:

- Modify: `core/tracker/client/src/main/kotlin/lava/tracker/client/LavaTrackerSdk.kt`

Steps:

- [ ] **Step 1: Signature + body.**

```
fun streamMultiSearch(
    request: SearchRequest,
    providerIds: List<String>,
    page: Int = 0,
```

```
): Flow<MultiSearchEvent> = channelFlow {
    val statuses = mutableMapOf<String, ProviderSearchStatus>()
    val resultsByProvider = mutableMapOf<String,
        List<TorrentItem>>()
    val displayNames = mutableMapOf<String, String>()

    for (id in providerIds) {
        launch {
            val descriptor = registry.list().firstOrNull {
        it.trackerId == id }
            if (descriptor == null) {
                send(MultiSearchEvent.ProviderFailure(id, "not
        registered"))
                statuses[id] = ProviderSearchStatus(id, id,
        ProviderSearchState.FAILURE, errorMessage = "not
        registered")
                return@launch
            }
            displayNames[id] = descriptor.displayName
            send(MultiSearchEvent.ProviderStart(id,
        descriptor.displayName))

            if (TrackerCapability.SEARCH !in
        descriptor.capabilities) {
                send(MultiSearchEvent.ProviderUnsupported(id))
                statuses[id] = ProviderSearchStatus(id,
        descriptor.displayName, ProviderSearchState.UNSUPPORTED)
                return@launch
            }

            val client = registry.get(id, MapPluginConfig())
            val feature =
        client.getFeature(SearchableTracker::class)
            if (feature == null) {
                send(MultiSearchEvent.ProviderUnsupported(id))
                statuses[id] = ProviderSearchStatus(id,
        descriptor.displayName, ProviderSearchState.UNSUPPORTED)
                return@launch
            }
            try {
                val result = feature.search(request, page)
                resultsByProvider[id] = result.items
                statuses[id] = ProviderSearchStatus(id,
        descriptor.displayName, ProviderSearchState.SUCCESS,
        resultCount = result.items.size)
                send(MultiSearchEvent.ProviderResults(id,
        result.items))
            } catch (t: Throwable) {
                statuses[id] = ProviderSearchStatus(id,
        descriptor.displayName, ProviderSearchState.FAILURE,
        errorMessage = t.message ?: "search failed")
                send(MultiSearchEvent.ProviderFailure(id,
        t.message ?: "search failed", t))
            }
        }
```

```
    }

    // Wait for all launches to complete; channelFlow's
        `awaitClose` is not needed
    // because the `launch`-es are children of this scope and
        complete before
    // channelFlow returns.
}.onCompletion { cause ->
    if (cause == null) {
        val deduped =
        DeduplicationEngine.deduplicate(resultsByProvider,
        displayNames)
        val unified = UnifiedSearchResult(
            query = request.query,
            page = page,
            items = deduped,
            totalPages = 1,
            providerStatuses = statuses.values.toList(),
        )
        emit(MultiSearchEvent.AllProvidersDone(unified = unified))
    }
}
```

Note: the local mutable maps captured by the inner launches are safe in coroutine-scope mutation (single-threaded confined to the parent's dispatcher) because `channelFlow` runs the producer on a single dispatcher unless the user opts into `flowOn`. If we ever move to `flowOn(Dispatchers.IO.limitedParallelism(...))`, swap the maps for `ConcurrentHashMap`.

- [ ] **Step 2: Compile.**

Task 4 — Delete `ActiveTrackerSection` + related state/action

The Phase C-added affordance is dead post-Phase-D. Delete with `@Deprecated` markers on the SDK API surface it called.

Files:

- Delete: `feature/provider_config/src/main/kotlin/lava/provider/config/sections/ActiveTrackerSection.kt`
- Modify: `feature/provider_config/src/main/kotlin/lava/provider/config/ProviderConfigState.kt` — remove `activeTrackerId` field
- Modify: `feature/provider_config/src/main/kotlin/lava/provider/config/ProviderConfigAction.kt` — remove `MakeActive`

- Modify: `feature/provider_config/src/main/kotlin/lava/provider/config/ProviderConfigViewModel.kt` — remove the `MakeActive` branch + the `activeTrackerId` field assignment in `onCreate`
- Modify: `feature/provider_config/src/main/kotlin/lava/provider/config/ProviderConfigScreen.kt` — remove `ActiveTrackerSection` import + invocation
- Modify: `core/tracker/client/src/main/kotlin/lava/tracker/client/LavaTrackerSdk.kt` — mark `activeTrackerId()` and both `switchTracker` overloads `@Deprecated("Use multiSearch/streamMultiSearch — single-active-tracker semantics removed by SP-4 Phase D")`.

Steps:

- [ ] **Step 1: `git rm` ActiveTrackerSection.kt.**
- [ ] **Step 2: Edit each of the 5 modified files** to drop the corresponding lines.
- [ ] **Step 3: Compile both feature modules.**

Task 5 — `SearchResultViewModel` consumes `streamMultiSearch` when API is not configured

Files:

- Modify: `feature/search_result/src/main/kotlin/lava/search/result/SearchResultViewModel.kt`

Steps:

- [ ] **Step 1: Branch in `onCreate`.**

```
onCreate = {
    observeFilter()
    when {
        mutableFilter.value.providerIds == null ->
        observePagingData()
        currentEndpointIsGoApi() ->
        observeSseSearch(mutableFilter.value)
        else -> observeStreamMultiSearch(mutableFilter.value)
    }
}
```

- [ ]

Step 2: Helper `currentEndpointIsGoApi()` reads `observeSettingsUseCase().first().endpoint is Endpoint.GoApi`.

- [ ] **Step 3: New private `observeStreamMultiSearch(filter)`** consumes `sdk.streamMultiSearch(...)`. On each event:
  - `ProviderStart(id, name)` → reduce `searchContent.activeProviders` adding/updating the provider row to `SEARCHING`.
  - `ProviderResults(id, items)` → append to `searchContent.items` + mark provider `DONE`.
  - `ProviderFailure(id, reason, _)` → mark provider `ERROR(reason)`.
  - `ProviderUnsupported(id)` → mark provider `UNSUPPORTED`.
  - `AllProvidersDone(unified)` → final reduce with the deduplicated unified result.

Reuse the existing `handleSseEvent`-style state-reduce logic; refactor the shared parts into a private helper `applyProviderEvent(state, event): SearchPageState`.

- [ ] **Step 4: Compile.**

Task 6 — Cancellation + backpressure unit tests

Files:

- Create: `core/tracker/client/src/test/kotlin/lava/tracker/client/LavaTrackerSdkParallelSearchTest.kt`

Steps:

- [ ] **Step 1: Parallel fan-out timing test.** Fake 3 providers with 500ms latency each. Assert `multiSearch(...)` completes in < 700ms (max latency + overhead), not 1500ms (sum of latencies).

- [ ] **Step 2: Cancellation test.** Start `streamMultiSearch(...)` collection; cancel after first `ProviderStart` event arrives; assert remaining provider coroutines are cancelled (verify by injecting a fake `SearchableTracker` whose `search` is a `suspendCancellableCoroutine` and asserting the cancellation handler ran).

- [ ]

Step 3: Backpressure test. Inject 5 fake providers that each emit instantly; use a collector with `delay(50)` between collects; assert all `ProviderStart` + `ProviderResults` + `AllProvidersDone` events are received in order (no drops, no reorderings).

- [ ] **Step 4: Bluff-Audit rehearsal.**
  - For (1): Mutate `multiSearch` body to `for (id in providerIds) { runOneProvider(...) }` (sequential). Confirm test fails with "took 1500ms, expected < 700ms".
  - For (2): Remove the `coroutineScope { }` wrapper so children are unstructured. Confirm cancellation test fails — child coroutines outlive the parent cancellation.
  - For (3): Change `channelFlow` buffer to `Channel.UNLIMITED` (no backpressure). Confirm test still passes for this scenario but document that the change weakens the contract; the test's purpose is to lock the SUSPEND policy. (For a falsifiable test of backpressure ITSELF, mutate the collector to NOT delay and assert that the emissions happen before the delay — but this is testing the test, not the SUT. Stick with the order-preservation assertion as the primary signal.)

  Record all three mutations + observed-failures in commit body.

- [ ] **Step 5: Compile + run tests.** `./gradlew :core:tracker:client:testDebugUnitTest`.

Task 7 — CONTINUATION update + commit + push

Steps:

- [ ] **Step 1: Update CONTINUATION.md** §0 with Phase D delivery.
- [ ] **Step 2: Local CI gate.** `bash scripts/check-constitution.sh` + the spotless/test runs.
- [ ] **Step 3: Commit** with:
  - Bluff-Audit stamps for `LavaTrackerSdkParallelSearchTest.kt` (3 mutations).
  - Notes on `@Deprecated`-marked API surfaces and the migration target.
  - `Co-Authored-By` trailer.

- [ ] **Step 4: Push to github + gitlab.** Verify SHA convergence.

Risk register

Risk	Mitigation
`channelFlow`'s `send` from inner `launch` can deadlock if the collector is slow + the producer holds a mutex	Avoid mutex inside the producer; rely on coroutine-confinement of the local maps (single dispatcher). Tested by Task 6 Step 3.
The legacy paging path still calls `LavaTrackerSdk.activeTrackerId()`	Deprecation marker is non-breaking; legacy callers continue to compile. Full removal is a future phase post-paging-migration.
`streamMultiSearch` emits `AllProvidersDone` via `onCompletion` — if the collector cancels mid-stream, `AllProvidersDone` is NOT emitted	Intentional. The consumer's `observeStreamMultiSearch` ViewModel logic does not depend on `AllProvidersDone` for final state; it incrementally reduces on each `ProviderResults` event. `AllProvidersDone` is the "all OK, here's the deduplicated summary" event for non-streaming callers.
Phase D commit and Phase C commit both reference `ActiveTrackerSection` — risk of merge surprise	Phase D ships only on master, ahead of Phase C in linear history. No branching surprise.
Cancellation test may flake on slow CI	Use generous timeouts (1000ms cancellation-propagation window).

Deliverable summary

A single commit (or 2-3 if grouping by domain) that:

1. Adds `MultiSearchEvent` + parallel `multiSearch` + `streamMultiSearch` to `LavaTrackerSdk`.
2. Wires `SearchResultViewModel` to consume the new flow when the Go API is not configured.

3. Deletes `ActiveTrackerSection` + related state/action; marks `LavaTrackerSdk.activeTrackerId()` + `switchTracker()` `@Deprecated`.
4. Adds 3 unit tests (parallel-timing, cancellation, backpressure) with Bluff-Audit stamps.
5. Documents C32 + C33 as Compose UI Challenges with KDoc rehearsal protocols (operator-bound execution).
6. Updates CONTINUATION.md.
7. Lands on both mirrors with verified SHA convergence.

Estimated session time: 1-2 hours of execution + the three unit-test mutation rehearsals (~3 minutes each). The Compose UI Challenges C32 + C33 are owed for operator execution before next tag, not for this implementation commit.